VAS

VIP Arrival System

Technical Gap Analysis

Phase 1 (Andrew) → MVP Operational System

What exists • What's missing • How to build it

Version 1.0 | March 2026 | Confidential

Part A: What Andrew Built (Phase 1)

Phase 1 delivered a clean, working backend engine. Here is exactly what exists in the codebase today:

A1. Technology Stack

Layer	Technology	Status
Runtime	`NestJS + TypeScript`	Production-grade framework
Database	`PostgreSQL + Prisma ORM`	Indexed, seeded, migrations
Infrastructure	`Docker + Docker Compose`	PostgreSQL + pgAdmin containers
Documentation	`Swagger/OpenAPI auto-gen`	Interactive API docs at /docs
Auth	`Single API key (header)`	x-api-key, env variable

A2. Working API Endpoints

Method	Endpoint	Function	Output
`POST`	`/api/ingress/plate-reads`	Receive plate reading from camera webhook	PlateRead + 3 AuditLog entries
`GET`	`/api/feed`	Query readings with cursor pagination + filtering	items[] + nextSince cursor
`POST`	`/api/vips`	Add new VIP (plate + name)	VIP record, 409 if duplicate
`GET`	`/api/vips`	Search VIPs by plate	VIP list
`GET`	`/health`	Health check (public, no auth)	{ status: "ok" }

A3. Database Schema (4 Tables)

Table	Key Columns	Notes
`PlateRead`	`id, plateRaw, plateNormalized, readAt, receivedAt, cameraId, confidence, snapshotUrl, isVip, matchType, vipId, rawPayload`	Indexed on plateNormalized, cameraId, receivedAt
`VIP`	`id, plateNormalized (unique), name, active`	10 seed records
`Camera`	`id, name, location`	3 seed cameras
`AuditLog`	`id, plateReadId, eventType, meta, createdAt`	3 events per read: received, normalized, matched

A4. Current Processing Pipeline

When a camera sends a webhook POST, the system executes this exact sequence:

```
Camera POST → Validate input → Verify camera exists → Normalize plate
  → Match vs VIP list (exact match) → Create PlateRead record
  → Create 3 AuditLog entries (received, normalized, matched)
  → Return response { id, plateNormalized, isVip, matchType }
```

Important: This is a synchronous request-response cycle. No queue, no async workers, no real-time push.

A5. Strengths of Phase 1 Code

- Clean modular NestJS architecture — easy to extend
- Prisma ORM protects against SQL injection
- Proper database indexes on key lookup fields
- Complete audit trail (3 events per read)
- Request tracking with unique IDs
- Input validation via class-validator
- Swagger auto-documentation
- Docker Compose for easy local development

A6. Known Weaknesses (Andrew's Report)

Issue	Risk	Status
Single API key, no rotation	Cannot revoke without redeploy	Phase 2 Security Sprint
No rate limiting	Vulnerable to DOS	Phase 2 Security Sprint
CORS open to all origins	Any domain can call API	Phase 2 Security Sprint
No unit/e2e tests	Regressions undetected	Phase 2 Security Sprint
console.log only logging	No structured log analysis	Phase 2 Security Sprint
No caching	VIP list queried every read	Future optimization
No monitoring	Blind to production issues	Future

Part B: Gap Analysis — Phase 1 vs MVP Requirements

The MVP operational system requires capabilities far beyond what Phase 1 delivers. This section maps every gap precisely.

B1. Feature-by-Feature Gap Matrix

Feature	Phase 1	MVP	Gap Description
Webhook receives plate	✓	✓	EXISTS. Add HMAC+nonce+IP allowlist validation layer on top.
Plate normalization	✓	✓	EXISTS. Works as-is. May need Saudi plate format tuning.
VIP exact matching	✓	✓	EXISTS. Core logic reusable. Need to scope it to active event.
Audit trail	✓	✓	EXISTS. Extend event types for APPROACHING/ARRIVED/CONFIRMED.
Feed API (polling)	✓	✓	EXISTS. Keep as fallback. Primary will be SSE real-time.
APPROACHING state	✗	✓	NEW. Camera 1 triggers approach alert. Needs VIPSession entity.
ARRIVED state	✗	✓	NEW. Camera 2 triggers arrival alert. State transition logic.
CONFIRMED state	✗	✓	NEW. Gate staff visual confirmation. Human-in-the-loop.
SSE real-time push	✗	✓	NEW. Ticket-based SSE. 5 concurrent connections max.
Session-based auth + RBAC	✗	✓	NEW. Replace API key with sessions + 3 roles.
Event management	✗	✓	NEW. Event entity with time window, status, VIP scope.
Camera role assignment	✗	✓	NEW. Camera has role: approach or gate. Determines alert type.
Guest card / profile	✗	✓	NEW. Extend VIP with company, protocol level, notes, photo.
Operations Dashboard	✗	✓	NEW. Full frontend. Alert feed, VIP mgmt, camera status.
Monitoring View	✗	✓	NEW. Read-only dashboard for security/protocol.
Gate confirmation UI	✗	✓	NEW. Simplified tablet UI. Confirm/reject buttons only.
Deduplication	✗	✓	NEW. Idempotency key: event_id+camera_id+plate+ts_bucket(3s).

Queue + workers	✗	✓✓	NEW. Async processing for 10+ cars/min burst.
Retroactive matching	✗	✓✓	NEW. VIP added after plate captured → 60min lookback match.
Sound alerts	✗	✓✓	NEW. Approach sound vs arrival sound. Frontend feature.
Edge/Local Mode	✗	✓✓	NEW. Standalone local system: SQLite+matching+alerts+dashboard.
HMAC webhook security	✗	✓✓	NEW. Replace simple API key with HMAC+nonce+TLS+IP allowlist.

Summary: 5 features reusable from Phase 1, 17 features must be built new.

B2. Database Schema Gap

Phase 1 has 4 tables. MVP needs 9. Here is what must be added:

Table	Status	What Changes
`PlateRead`	Modify	Rename to PlateEvent. Add: idempotency_key, is_late, event_id. Keep existing columns.
`VIP`	Modify	Add: company, protocol_level, notes, photo_url, event_id. Multi-plate via Plate table.
`Camera`	Modify	Add: role (approach|gate), last_seen, event_id. Keep name, location.
`AuditLog`	Extend	Add event types: VIP_ADDED, APPROACHING, ARRIVED, CONFIRMED, LATE_READ.
`Event`	NEW	id, name, start_time, end_time, status, event_window (8hr default).
`Plate`	NEW	id, plate_number, vip_id. Allows multiple plates per VIP.
`VIPSession`	NEW	id, vip_id, event_id, status, approach_at, arrived_at, confirmed_at, confirmed_by.
`User`	NEW	id, name, email, role (admin|operator|manager), password_hash.
`UserPermission`	NEW	id, user_id, permission (CAN_ADD_VIP), granted_by, event_id.

B3. Processing Pipeline Gap

Phase 1 (current): Synchronous request-response

```
Camera POST → Validate → Normalize → Match → Store → Return
```

MVP (required): Async queue-based with real-time push

```
Camera POST → HMAC validate → Deduplicate (idempotency key)
  → Enqueue to job queue
  → Worker picks up: Normalize → Store PlateEvent
  → Match vs active VIP list for current event window
  → Determine camera role (approach vs gate)
  → Create/update VIPSession with state transition
  → If retroactive: search PlateEvents from last 60 min
  → Emit SSE event (ALERT_CREATED / VIP_ARRIVED)
  → Dashboard plays sound + renders alert
```

Key architectural shift: Phase 1 is synchronous and stateless. MVP requires async processing, stateful sessions, and real-time push.

Part C: Step-by-Step Build Plan

Each sprint builds on the previous one. No sprint can be skipped.

Sprint 0: Foundation Fixes (2-3 days)

Goal: Secure the existing codebase and prepare for new architecture.

Backend tasks:

1. Add Helmet + security headers
2. Restrict CORS to specific origins
3. Add rate limiting (100 req/min per IP)
4. Replace console.log with structured logging (Pino/Winston)
5. Add body size limit (1MB)
6. Set up test framework (Jest) with basic smoke tests

Output: Secure, logged, testable codebase ready for extension.

Sprint 1: Core Schema + Auth Overhaul (5-7 days)

Goal: New database schema, session auth, RBAC, and event management.

Database migration:

1. Create Prisma migration: add Event, Plate, VIPSession, User, UserPermission tables
2. Modify VIP table: add company, protocol_level, notes, photo_url, event_id
3. Modify Camera table: add role (approach|gate), last_seen, event_id
4. Rename PlateRead to PlateEvent, add idempotency_key, is_late, event_id
5. Update seed data: create default admin user, sample event, camera roles

Auth module:

6. Replace API key auth with session-based auth (express-session + connect-pg-simple)
7. Build login endpoint: POST /auth/login (email + password)
8. Build RBAC guard: @Roles(ADMIN, OPERATOR, MANAGER) decorator
9. Build user management: POST /users (admin-only), GET /users
10. Build event management: POST /events, GET /events/active, PATCH /events/:id

Output: Multi-user system with roles, events, and extended VIP profiles.

Sprint 2: State Machine + Webhook Security (5-7 days)

Goal: VIPSession lifecycle, deduplication, HMAC webhook security, and camera health.

Webhook security:

1. Add HMAC signature validation middleware on ingress endpoint
2. Add nonce tracking (prevent replay attacks)

3. Add IP allowlist middleware (camera source IPs)

Deduplication:

4. Generate idempotency key = hash(event_id + camera_id + plate + ts_bucket_3s)
5. Check idempotency key before processing; reject duplicates with 200 OK

State machine:

6. VIPSession service: create session on first VIP match
7. State transitions: REGISTERED → APPROACHING (cam1) → ARRIVED (cam2) → CONFIRMED (gate staff) → COMPLETED
8. Camera role detection: check camera.role to determine which state to transition to
9. Late read handling: if plate arrives outside event window, tag LATE, log, no alert
10. Retroactive matching: POST /vips triggers PlateEvent search in last 60 min lookback

Camera health:

11. Update camera.last_seen on every received webhook; expose GET /cameras/health

Output: Full state machine, secure webhook intake, deduplication, retroactive matching.

Sprint 3: SSE Real-Time + Queue (4-5 days)

Goal: Real-time push to dashboards, async job queue for burst handling.

SSE module:

1. POST /realtime/ticket — generate short-lived UUID ticket (60s expiry)
2. GET /realtime/stream?ticket=<uuid> — SSE endpoint, validates ticket, keeps connection
3. Event types: ALERT_CREATED, VIP_ARRIVED, VIP_ADDED, VIP_CONFIRMED, CAMERA_HEALTH
4. Backpressure rule: ALERT_CREATED and VIP_ARRIVED must never be dropped
5. Connection limit: max 5 concurrent SSE connections

Job queue:

6. Add BullMQ (Redis-backed) or in-memory queue for plate processing
7. Webhook endpoint enqueues job immediately, returns 202 Accepted
8. Worker processes: normalize → dedup → match → state transition → SSE emit

Output: Real-time push to all dashboards, handles 10+ cars/min burst without data loss.

Sprint 4: Frontend Dashboards (7-10 days)

Goal: Three user interfaces — Operations Dashboard, Monitoring View, Gate UI.

Operations Dashboard (Operator/Admin):

- Real-time alert feed with SSE subscription
- Sound differentiation: approach tone vs arrival tone
- VIP management: add/edit guests, plates, company, protocol level, notes, photo
- Event management: create event, set time window, activate/close
- Camera status panel: last_seen per camera, health indicator
- Guest card popup on alert click: full profile with context
- Alert list: chronological, persistent until acknowledged

Monitoring View (Manager, read-only):

- Same SSE feed, same alerts, no edit controls
- Overview of all active VIP sessions
- Clean, read-only interface for security/protocol staff

Gate Confirmation UI (tablet/mobile):

- Simplified interface for gate staff
- Shows only ARRIVED alerts relevant to this gate
- One large confirm button + one reject button
- Responsive design (tablet-first)
- Login: simple username + pin code

Output: Three working interfaces connected to live SSE stream.

Sprint 5: Edge/Local Mode (5-7 days)

Goal: Standalone local system for events with no internet.

1. Package backend as standalone executable or local Docker container
2. Replace PostgreSQL with SQLite for local storage
3. Pre-load VIP list from cloud before event (manual sync)
4. Local matching + local alerts + local dashboard (same UI)
5. Post-event sync: manual trigger pushes all data to cloud (500 records/batch)
6. Idempotency keys prevent duplicate records during re-sync

Output: VAS works in locations with zero internet. All data syncs later.

Part D: Summary

D1. Effort Estimate by Sprint

#	Sprint	Duration	Depends On
0	Foundation Fixes (Security)	2-3 days	—
1	Schema + Auth + RBAC + Events	5-7 days	Sprint 0
2	State Machine + Webhook Security	5-7 days	Sprint 1
3	SSE Real-Time + Queue	4-5 days	Sprint 2
4	Frontend Dashboards (3 UIs)	7-10 days	Sprint 3
5	Edge/Local Mode	5-7 days	Sprint 4
Total		**28-39 working days**	

D2. What Andrew's Code Gives Us

- Working NestJS project structure — modules, controllers, services, DTOs
- Prisma ORM setup with migrations and seed — ready to extend
- Plate normalization logic — reusable as-is
- VIP matching engine — core logic reusable, needs event-scoping
- Audit trail pattern — extend with new event types
- Docker Compose infrastructure — add Redis for queue
- Swagger documentation — automatically updates with new endpoints

D3. What Must Be Built From Scratch

- VIPSession state machine (REGISTERED → APPROACHING → ARRIVED → CONFIRMED → COMPLETED)
- Session-based auth replacing API key + RBAC with 3 roles
- Event management module (create, activate, close events)
- HMAC + nonce + IP allowlist webhook security
- Deduplication via idempotency keys
- SSE real-time module with ticket-based auth
- Job queue for async burst processing
- Retroactive matching on VIP addition
- Three frontend interfaces (Ops Dashboard, Monitoring View, Gate UI)
- Edge/Local standalone mode with SQLite + manual sync
- Camera role assignment and health monitoring

— End of Document —